
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 10 — Chapter 16: The Complete SVM Code Explained Line by Line
Preprocessing Pipeline | Training, Prediction, Saving & Loading the Model

PART 1 — From Feature Matrix to Trained Model

Chapter 16 — The Complete SVM Code Explained Line by Line Part 1 — From Feature Matrix to Trained Model Before Starting Congratulations. Until now we have studied everything from theoretical and engineering perspectives. Now we'll finally study our actual code. This chapter is extremely important because after finishing it, you should be able to explain every important line without memorizing it. Instead, you'll understand why every line exists. Where Are We? Our project has reached this stage. **LTspice** — CSV Files.

Feature Extraction

17 Features

Machine Learning Code ← We are here Now the computer already has the feature matrix.

Step 1 — Loading the Dataset

The first task is reading the extracted features. Conceptually our code looks like

```
data = pd.read_csv(...)
```

Question. What happens here? Nothing related to

Machine Learning

yet. The program is simply reading the dataset from disk into memory.

In Our Project

This dataset already contains our 17 extracted features plus the fault label. Something like

5.21	3.44	2.11	...	4.98	RC_Normal
4.90	3.20	1.95	...	0.001	RC_Open

Every row represents one waveform.

PROFESSOR ASKS

Why do we load the dataset before training?

EXCELLENT ANSWER

The machine learning algorithm requires numerical feature vectors and corresponding labels as input. Therefore, the extracted feature dataset must first be loaded into memory.

Step 2 — Separating Features and Labels

Now the program splits the dataset into two parts. Question. Why? Because

Machine Learning

must know which columns are inputs and which column is the correct answer. Conceptually our code looks like $X = \text{data}[\text{features}]$ $y = \text{data}["\text{Label}"]$ Question. What is X? Answer. The feature matrix. Contains Peak RMS Mean. DC Level Exactly 17 features. Question. What is y? Answer. The fault labels. Example RC_Normal RC_Open RL_Normal.

PROFESSOR ASKS

What are X and y?

EXCELLENT ANSWER

X contains the input feature vectors extracted from the waveform, while y contains the corresponding fault labels used during supervised learning. Why Use X and y? Almost every Machine Learning library expects exactly this format. **Inputs** — X. **Outputs** — y. Simple. Standard.

Step 3 — Train-Test Split

Question. Can we train using all the data? Yes. Question. Should we? No. Need an independent test set. Conceptually our code looks like `train_test_split()` Question. Why? Because we must evaluate the model using data it has never seen. Otherwise we cannot measure generalization.

Real Life Analogy

Imagine a football coach. Question. Should practice matches and the World Cup be the same? No. Training and evaluation must be separate. Exactly the same idea.

PROFESSOR ASKS

Why do we split the dataset?

EXCELLENT ANSWER

The dataset is divided into training and testing sets so that the model can be trained on one portion of the data and evaluated on previously unseen samples.

Step 4 — StandardScaler

Now one of the most important lines in the entire code. Many students simply memorize `StandardScaler()` without knowing why. Let's understand it. Question. Suppose our features look like **Peak** — 5.2. **Rise Time** — 0.00024. **Energy** — 480. Question. Are these on the same scale? No. Huge difference. Question. Will SVM treat them equally? No. Large-valued features may dominate the distance calculations. Need Scaling. What Does `StandardScaler` Do? It transforms every feature to approximately the same scale. Conceptually **Before** — 5.2. 480 0.00024 **After** — 0.8. 1.1 -0.4 Notice everything is now comparable.

PROFESSOR ASKS

Why did you use `StandardScaler`?

EXCELLENT ANSWER

SVM is sensitive to feature scales because it relies on distance calculations. `StandardScaler` normalizes the feature distributions so that no single feature dominates the learning process. Excellent.

Real Life Analogy

Suppose one exam is out of 10 marks. Another is out of 1000 marks. Question. Can you compare them directly? No. Need normalization. Exactly what `StandardScaler` does.

Important Point

Question. Did `StandardScaler` change the meaning of the features? No. It only changed their numerical scale. Peak is still Peak. Energy is still Energy. Nothing physical changed.

Step 5 — Fitting the Scaler

Students often confuse this step. Question. Should we calculate scaling parameters using the entire dataset? No. Only the training set. Conceptually `scaler.fit(X_train)` Question. Why? Because using test data would leak future information into training. That is called Data Leakage. Very important.

PROFESSOR ASKS

Why is the scaler fitted only on the training set?

EXCELLENT ANSWER

The scaler is fitted only on the training data to prevent information from the test set influencing the training process, thereby avoiding data leakage.

Step 6 — Transforming Both Sets

After learning the scaling parameters, the scaler transforms both datasets. `X_train = scaler.transform(X_train)` `X_test = scaler.transform(X_test)` Notice something. The scaler learns only once. Then applies the same transformation to both datasets. Why Is This Important? Suppose we calculated a different scaling for the test set. Question. Would training and testing still use the same measurement system? No. That would be incorrect.

Our Pipeline Now

Everything we've studied fits together. **CSV Dataset** — X. y

Train-Test Split

StandardScaler

Scaled Features

SVM Exactly what our project does.

COMMON MISTAKES

MISTAKE: [X] Scaling improves accuracy by itself. Wrong. It improves learning conditions, not the information contained in the features.

MISTAKE: [X] Fit the scaler using all data. Wrong. Only training data.

MISTAKE: [X] Scaling changes the physics of the waveform. Wrong. Only the numerical representation changes.

REVISION NOTES

Remember
the complete preprocessing pipeline.
Load Dataset — Read Features.
Separate — X and y.
Train-Test Split — Separate Learning and Evaluation.
StandardScaler — Normalize Feature Scale.
Transform — Prepare for SVM.

MEMORY TRICK

Remember this sentence. "Load, Split, Scale, Train." Four words. Entire preprocessing pipeline.

KNOWLEDGE CHECK

Level 1

- What is X?
- What is y?
- Why do we split the dataset?
- Why do we use StandardScaler?
- What is data leakage?

Level 2

- Why is SVM sensitive to feature scales?
- Why is the scaler fitted only on training data?
- Why are the same scaling parameters used for the test set?
- What problems occur if scaling is skipped?
- How does preprocessing improve the reliability of the classifier?

Professor Level

- Explain the complete preprocessing pipeline used before SVM training.
- Why is StandardScaler particularly important for SVM?
- What is data leakage and how does your implementation prevent it?
- Why are X and y separated before training?
- Explain the role of preprocessing in the overall machine learning pipeline.
- End of Chapter 16 (Part 1)
- Congratulations.
- You now fully understand the preprocessing section of your machine learning code.
- Notice something important:
- Until this point, the SVM has still not started learning.
- Everything we've done so far—loading the dataset, separating features and labels, splitting the data, and scaling the features—is preparation.
- Only after this stage does the actual SVM training begin.

Author's Note The next continuation will explain the core SVM code line by line, including: Creating the SVC object. Every parameter inside SVC(.). Calling fit(). Calling predict(). Saving the trained model. Loading the model later for inference. Why these steps are necessary in your dashboard. This chapter will directly connect your training code with the deployment stage of your project.

PART 2 — SVM Training, Prediction and Model Saving

Chapter 16 — The Complete SVM Code Explained Line by Line Part 2 — SVM Training, Prediction and Model Saving Before Starting

In Part 1

we completed the preprocessing. Our pipeline currently looks like this. **Dataset** — X, y

Train-Test Split

StandardScaler

Scaled Features

Question. Has

Machine Learning

started? No. Everything until now was preparation. Now the actual learning begins.

Step 7 — Importing SVC

The first line related to

Machine Learning

is conceptually

```
from sklearn.svm import SVC
```

Question. What is SVC? SVC means Support Vector Classifier. It is Scikit-learn's implementation of

Support Vector Machine

for classification problems.

PROFESSOR ASKS

What is SVC?

EXCELLENT ANSWER

SVC is Scikit-learn's implementation of the Support Vector Machine algorithm used for classification tasks.

Step 8 — Creating the SVM Model

Now our code creates the classifier. Conceptually it looks like

```
model = SVC(...)
```

Question. Has the model learned anything? No. This line only creates an empty classifier. Think of it like opening a blank notebook. Nothing has been written yet.

Real Life Analogy

Suppose you buy a brand new calculator. Question. Does it already know your project? No. It is ready to work, but nothing has happened yet. Exactly the same.

Step 9 — Choosing the Kernel

Inside `SVC()` one important parameter is `kernel="rbf"` Question. Why did we use RBF? Because our fault classes are not perfectly linearly separable. The RBF kernel creates nonlinear decision boundaries that can separate complex feature relationships. We already studied this in detail.

PROFESSOR ASKS

Why RBF instead of Linear?

EXCELLENT ANSWER

The extracted features exhibit nonlinear relationships. The RBF kernel allows the SVM to learn nonlinear decision boundaries, making it more suitable for analog circuit fault classification.

Step 10 — C Parameter

Another parameter inside `SVC(.)` is `C =`. Question. Did we choose this value manually? No. `GridSearchCV` selected the best value. Remember Chapter 13. Question. What does `C` control? The balance between training accuracy and generalization.

Small C

Larger Margin

More tolerance to mistakes.

Large C

Smaller Margin

Fewer training errors.

PROFESSOR ASKS

What does C control?

EXCELLENT ANSWER

The `C` parameter controls the trade-off between maximizing the margin and minimizing training classification errors.

Step 11 — Gamma

Another important parameter is `gamma =`. Again chosen by `GridSearchCV`. Question. What does `Gamma` control? It controls how much influence each training sample has on the decision boundary.

Small Gamma

Smooth boundary.

Large Gamma

Complex boundary.

PROFESSOR ASKS

What is Gamma?

EXCELLENT ANSWER

`Gamma` determines the influence of individual training samples on the decision boundary when using the RBF kernel.

Step 12 — Training the Model

Now the most important line in the entire Machine Learning code. `model.fit(X_train, y_train)` Question. What happens here? Everything. This is where learning actually begins. Inside `fit()` SVM looks at every training sample. Example Peak 5.12 RMS 3.41. DC Level **4.98** — RC_Normal. Then another sample. Then another. Thousands of samples. Eventually it finds the optimal hyperplane that separates the fault classes. Question. Does `fit()`

```
return
```

accuracy? No. Does it

```
return
```

predictions? No. It creates a trained model.

PROFESSOR ASKS

What happens during fit()?

EXCELLENT ANSWER

During fit(), the SVM analyzes the training feature vectors and corresponding labels to learn the optimal decision boundaries separating the fault classes. What Does the Model Learn? Many students think the model stores the dataset. Wrong. It learns mathematical relationships. Specifically, it stores information such as Support Vectors Decision Boundaries Hyperplane Parameters Kernel Information It does NOT store LTspice waveforms. It does NOT memorize CSV files.

Step 13 — Prediction

Now the model faces new data. Conceptually, our code uses `predictions = model.predict(X_test)` Question. What is X_test? Feature vectors the model has never seen before. Exactly what we want. Example Suppose a test sample contains Peak 4.92 RMS 3.28. DC Level 0.001 Question. What happens? SVM compares this feature vector with the learned decision boundaries and predicts RC_Open Simple.

PROFESSOR ASKS

What does predict() do?

EXCELLENT ANSWER

`predict()` applies the trained SVM model to previously unseen feature vectors and returns the predicted fault classes.

Step 14 — Evaluating Predictions

Now we compare

Predicted Labels

with

Actual Labels

This produces Accuracy Confusion Matrix Precision Recall F1 Score Exactly what we studied in Chapter 14. Notice Evaluation happens after Prediction.

Step 15 — Saving the Model

Now one very useful step. Question. Suppose training takes 10 minutes. Question. Should we train again every time the dashboard opens? No. Instead we save the trained model. Conceptually, our code looks like `joblib.dump(model,.)` or `pickle.dump(.,)` depending on the implementation. Question. Why save the model? Because training is expensive. Loading is fast.

Real Life Analogy

Imagine writing a book. Question. After finishing, do you rewrite the book every time you want to read it? No. You save it. Exactly the same.

PROFESSOR ASKS

Why do we save the trained model?

EXCELLENT ANSWER

The trained model is saved so that it can be reused for future predictions without repeating the entire training process.

Step 16 — Loading the Model

Later, when the dashboard starts, we simply load the saved model. Conceptually, `joblib.load(.)` Now the model is immediately ready for prediction. No retraining is required.

Dashboard Flow

This is exactly what happens inside our application. User

Uploads Waveform

Feature Extraction

StandardScaler — Load Saved SVM. Prediction

Display Fault

Notice there is no training inside the dashboard. Only prediction.

Why This Is Efficient

Training — Done once. **Prediction** — Done. thousands of times. Exactly how real Machine Learning systems work.

Connection to Our Project

Our complete Machine Learning pipeline is now Dataset

Train-Test Split

StandardScaler — SVC. fit()

Trained Model

Save Model

Load Model

predict()

Fault Diagnosis

Everything fits together.

COMMON MISTAKES

MISTAKE: [X] predict() trains the model. Wrong.

MISTAKE: [X] fit() predicts new data. Wrong.

MISTAKE: [X] The dashboard trains the SVM. Wrong. The dashboard only loads the already-trained model.

MISTAKE: [X] Saving the model changes its accuracy. Wrong. It only stores the trained model.

REVISION NOTES

Remember
the workflow.
Create — SVC.
Train — fit().
Predict — predict().
Save — joblib.dump().
Load — joblib.load().
Dashboard — Inference Only.

MEMORY TRICK

Remember this sentence. "Create → Fit → Save → Load → Predict." Five words. Entire deployment pipeline.

KNOWLEDGE CHECK

Level 1

- What is SVC?
- What does fit() do?
- What does predict() do?
- Why do we save the model?
- Why doesn't the dashboard train the SVM?

Level 2

- Explain the complete SVM workflow in your project.
- Why is the model trained only once?
- Why is model loading much faster than training?
- What information is stored in the trained SVM?
- How does the dashboard use the trained model?

Professor Level

- Explain the complete training and deployment pipeline used in your project.
- Why is it important to separate training from inference?
- How does model persistence improve the usability of your application?
- Why is saving the trained model considered standard practice in machine learning deployment?
- Describe how a user interacts with the dashboard from waveform upload to fault prediction.
- End of Chapter 16 (Part 2)
- Congratulations.
- You now understand the entire SVM implementation, from creating the model to training, prediction, saving, and deployment.
- At this point, you can confidently explain the complete machine learning pipeline in your project, not just the theory behind SVM.

Author's Note The next chapter will move to the ANN (Artificial Neural Network) section of your project. We'll explain: Why we implemented an ANN in addition to SVM. The complete ANN architecture. Every layer and activation function. Training process. Loss function. Optimizer. Epochs and batch size. Why ANN performed the way it did compared to SVM. A detailed comparison between SVM and ANN for your analog fault diagnosis project. This chapter will complete the machine learning portion of the Project Bible before we move on to the dashboard and deployment.